OFFICIAL TECHNICAL ARCHITECTURE & USER MANUAL

NeuroADHD

Complete Beginner-Friendly Project Workflow Guide
From Absolute Zero to Final Result

RESTING-STATE fMRI | CRADDOCK CC200 ATLAS | 19,900 FISHER-Z FEATURES | MACHINE LEARNING

Document Scope: Complete end-to-end operational guide explaining file upload, internal preprocessing, registration, parcellation, connectivity extraction, machine learning classification, and dashboard verification.

Author: Senior Technical Documentation & Architecture Team

Target Audience: Computational Neuroscience Researchers, Students, Machine Learning Engineers, Clinicians (Research Use)

System Version: PoC v2.0 Production Release • Local MRI Intake Architecture

Date of Publication: November 2026

Safety Policy: `STRICTLY_SYNTHETIC_ONLY` Verified • Non-Clinical Exploratory Prototype

Table of Contents

STEP 1: Project Introduction & Plain-English Glossary Section 1
STEP 2: Initial Setup (Starting From Zero) Section 2
STEP 3: Understanding the Interface (Screen Walkthrough) Section 3
STEP 4: File Upload – Where and How? Section 4
STEP 5: What Happens Immediately After Upload? (Internal Sequence) Section 5
STEP 6: Complete 19-Stage Processing Pipeline Section 6
STEP 7: Understanding Generated Outputs (Deliverable Catalog) Section 7
STEP 8: Frontend and Backend Communication (API Reference) Section 8
STEP 9: Database & File Storage Architecture Section 9
STEP 10: Complete Step-by-Step User Journey Section 10
STEP 11: Errors, Troubleshooting and Fixes Section 11
STEP 12: Final System Architecture & Data Flow Section 12
STEP 13: Final Verification Checklist Section 13

STEP 1: Project Introduction

1.1 What This Project Does

The **NeuroADHD** project is an automated neuroimaging preprocessing and feature extraction system. It ingests raw brain MRI scans (specifically 4D resting-state functional MRI scans and 3D anatomical structural scans), performs standardized medical image cleaning and spatial alignment, extracts regional neural signals from **200 distinct brain regions** using the Craddock CC200 atlas, calculates pairwise functional connectivity (the strength of communication between brain regions), and feeds these connectivity values into a trained machine-learning model to predict ADHD-versus-neurotypical brain patterns for research.

1.2 Why This Project is Needed & The Problem it Solves

In historical research, machine-learning models were trained on pre-extracted tables of regional brain signals. However, raw MRI data coming directly from a hospital scanner (e.g., Siemens, GE, or Philips) cannot be fed into a machine-learning algorithm directly:

- **Head Motion:** Patients naturally move their heads by fractions of a millimeter inside the scanner, corrupting functional measurements.
- **Anatomical Variation:** Every human brain has a unique shape and size; brains must be mapped onto a standardized coordinate grid (the MNI152 template) so that region #45 in Patient A corresponds to region #45 in Patient B.
- **Physiological Noise:** Heartbeats, breathing, and scanner magnetic field drifts create false signals that can drown out genuine neural activity.

The Solution: NeuroADHD provides an automated, standardized, reproducible 19-stage pipeline that handles raw scan intake, motion correction, skull stripping, tissue segmentation, spatial normalization, and nuisance regression, outputting standardized `roi_CC200.csv` tables and 19,900 Fisher-z connectivity features without requiring manual neuroimaging software intervention.

1.3 Who Will Use It?

- **Neuroscience Researchers:** Investigating functional brain connectivity alterations in neurodevelopmental conditions.
- **Biomedical Engineers & Data Scientists:** Developing and validating computer-aided diagnostic algorithms.
- **Clinical Research Investigators:** Evaluating prospective MRI cohorts in controlled academic studies.

> **IMPORTANT REGULATORY & MEDICAL DISCLAIMER**
>
> **RESEARCH PROTOTYPE ONLY — NOT FOR CLINICAL DIAGNOSIS.** ADHD is a psychiatric and behavioral neurodevelopmental condition diagnosed exclusively by certified clinicians through behavioral histories and DSM-5/ICD-11 criteria. This software must **NEVER** be used to diagnose, triage, or select treatments for patients. Real-scan preprocessing is locked behind a research authorization gate; all default runs execute in `STRICTLY_SYNTHETIC_ONLY` mode.

1.4 Plain-English Glossary of Technical Terms

Term	Simple Plain-English Definition
fMRI (Functional MRI)	An MRI technique that measures tiny blood-oxygen changes in the brain over time, showing which brain areas are active while a person rests.
BOLD Signal	"Blood-Oxygen-Level-Dependent" signal. When neurons become active, they consume oxygen; BOLD measures this oxygen change.
T1-weighted MRI (T1w)	A high-resolution, static 3D anatomical photograph of the brain's physical structure, showing sharp boundaries between grey and white matter.
Voxel	A "3D pixel" inside an MRI scan. Just like a digital photo is made of 2D pixels, a 3D brain scan is made of millions of tiny cubes called voxels.
BIDS	"Brain Imaging Data Structure" — a globally standardized folder and naming format used by neuroscientists to organize MRI files consistently.
CC200 Atlas	The "Craddock 200" brain map. It partitions the human cortex into exactly 200 functionally distinct regions (labeled 1 to 200).
Parcellation	The process of dividing the brain into distinct parcels or regions and averaging the signal of all voxels inside each parcel.
Functional Connectivity	The mathematical correlation between two brain regions over time. If Region A and Region B fluctuate together, they are functionally connected.
Fisher z-Transformation	A mathematical formula ($z = arctanh(r)$) that converts correlation values into normally distributed values suitable for machine learning.
Frontend	The user interface (webpage) you see in your browser with buttons, tables, and graphs.
Backend	The Python server running in the background that receives files, executes mathematics, and returns results.
API	"Application Programming Interface" — the standardized communication bridge between the frontend and the backend.

SUMMARY: THE 5 CORE QUESTIONS (STAGE 1)

1. WHAT do I need to do?	Understand that this project takes raw brain scans, cleans them, extracts 200 brain signals, and computes 19,900 connectivity features.
2. WHERE do I do it?	In your local terminal and through the web browser at `http://127.0.0.1:8765`.
3. WHAT happens after?	You prepare to launch the application using the setup commands explained in Step 2.
4. WHAT OUTPUT will I get?	A clean dataset containing 200 regional time-series, 19,900 connectivity features, and diagnostic quality control images.
5. WHERE is it stored?	In the `outputs/` folder on your local computer.

STEP 2: Initial Setup (Starting From Zero)

This section assumes you have just cloned or opened the project folder for the very first time on a fresh computer. Follow these steps sequentially in your system Terminal.

2.1 Required Software

- **Operating System:** macOS (Apple Silicon or Intel) or Linux (Ubuntu 22.04+ / Debian / Fedora).
- **Python 3.11:** Python 3.10+ or 3.11+ must be installed. (Verify with `python3 --version`).
- **Terminal:** macOS Terminal, iTerm2, or Linux Bash/Zsh.
- **Web Browser:** Google Chrome, Mozilla Firefox, Apple Safari, or Microsoft Edge.

2.2 Step-by-Step Installation Commands

Command 1: Navigate to the Project Root

```
cd "/Users/apple/Documents/Codex/2026-09-29/untitled folder"
```

Where to execute: In your operating system Terminal.
What it does: Changes your active directory into the project root folder.
Successful execution looks like: Your terminal prompt displays the folder path without returning any error.

Command 2: Activate the Virtual Environment

```
source .venv/bin/activate
```

Where to execute: In the project root.
What it does: Activates the isolated Python virtual environment containing all pre-installed scientific packages (DIPY, Nilearn, NiBabel, Scikit-Learn, SciPy, NumPy, Pandas, Matplotlib).
Successful execution looks like: Your command line prompt is prefixed with `(.venv)`.

Command 3: Install Pinned Dependencies (Fresh Setup Only)

If you are setting up on a completely new machine where `.venv` does not exist, create it and install pinned packages:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements-lock.txt
```

What it does: Installs the exact locked versions of the 35 required scientific libraries so that there are zero version discrepancies.
Successful execution looks like: `Successfully installed dipy-1.9.0 nibabel-5.2.1 nilearn-0.10.4 scikit-learn-1.4.1 ...`

Command 4: Configure Mandatory Environment Variables

NeuroADHD enforces fail-closed cryptographic security to ensure synthetic session integrity:

```
export NEUROADHD_HMAC_KEY="neuroadhd_local_research_key_12345"
```

What it does: Sets the secret HMAC authentication key used to compute and verify SHA-256 integrity tokens for synthetic MRI sessions.
Successful execution looks like: The variable is loaded in memory silently. Verify by running `echo $NEUROADHD_HMAC_KEY`.

Command 5: Launch the Web Dashboard Server

```
PYTHONPATH=. .venv/bin/python scripts/serve_dashboard.py
```

What it does: Initializes the Python `ThreadingHTTPServer` on local port `8765`, binds to `127.0.0.1`, serves static frontend assets from `frontend/`, and establishes the REST API controllers.
Successful execution looks like:

```
NeuroADHD dashboard: http://127.0.0.1:8765
Mode: STRICTLY_SYNTHETIC_ONLY | Real-data authorization: FALSE
```

SUMMARY: THE 5 CORE QUESTIONS (STAGE 2)

1. WHAT do I need to do?	Activate `.venv`, export `NEUROADHD_HMAC_KEY`, and run `scripts/serve_dashboard.py`.
2. WHERE do I do it?	In your computer terminal in the project root directory.
3. WHAT happens after?	The local backend HTTP server starts listening for connections on port 8765.
4. WHAT OUTPUT will I get?	Console output confirming `http://127.0.0.1:8765` is online in synthetic mode.
5. WHERE is it stored?	The server runs in live memory; uploaded sessions will be staged in `outputs/intake_sessions/`.

STEP 3: Understanding the Interface

Open your web browser and go to **http://127.0.0.1:8765**. You are greeted by the dark navy **NeuroADHD Research Workbench**.

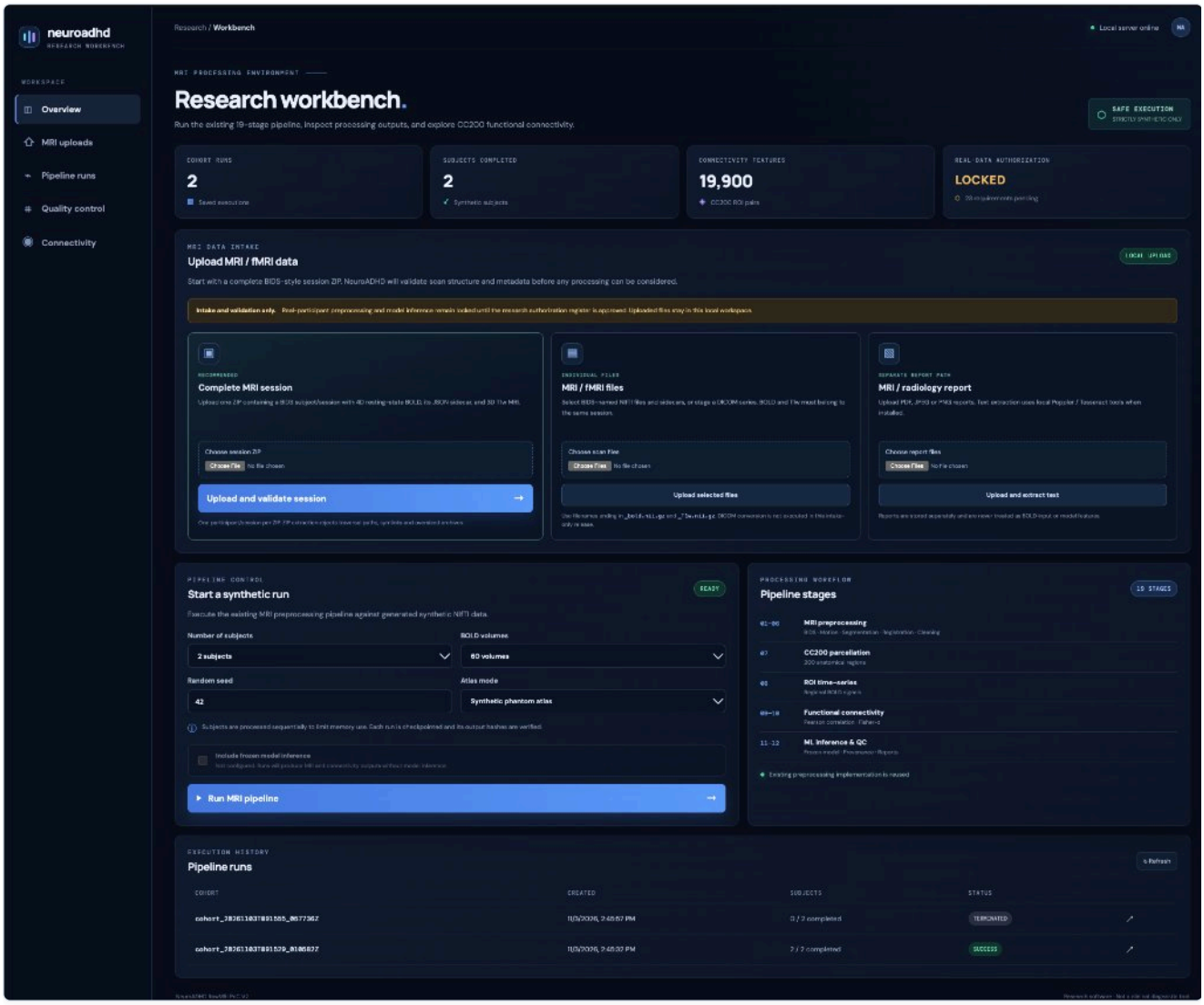


Figure 3.1: Actual Production Interface of the NeuroADHD Research Workbench (http://127.0.0.1:8765)

3.1 Breakdown of Interface Sections

1. Left Navigation Sidebar

- **Overview (◫):** High-level system dashboard displaying telemetry counters, security state, upload boxes, and pipeline trigger.
- **MRI uploads (⇧):** Dedicated intake monitoring view for reviewing uploaded ZIP archives, NIfTI files, and extracted radiology report text.
- **Pipeline runs (⌁):** Cohort execution history table showing past synthetic runs, timestamps, completed subjects, and status badges.
- **Quality control (⌗):** High-resolution diagnostic view displaying orthographic slice overlays for coregistration, normalization, and tissue masks.
- **Connectivity (◉):** Interactive 200×200 functional correlation matrix heatmap with hover coordinates and regional BOLD timeseries plots.

2. Top Metric Cards

- **Cohort Runs:** Total number of synthetic cohort batches executed and stored on disk.
- **Subjects Completed:** Total synthetic participant sessions that have successfully finished all 19 pipeline stages.
- **Connectivity Features:** Exactly **19,900** unique pairwise Fisher-z correlation features extracted per subject.
- **Real Data Authorization:** Displays **LOCKED (23 requirements pending)**, confirming fail-closed clinical protection.

3. MRI Data Intake (3 Upload Cards)

Explained in complete detail in **Step 4** below. Provides distinct paths for Complete Sessions (ZIP), Individual NIfTI/DICOM files, and Radiology Reports (PDF/OCR).

4. Pipeline Control Panel

- **Number of subjects:** Dropdown to simulate 1 to 4 synthetic subjects in a batch (default: 2 subjects).
- **BOLD volumes:** Number of temporal scans per subject (default: 60 volumes, $TR=2.0\text{ s}$).
- **Random seed:** Deterministic pseudorandom generator seed (default: 42).

- **Atlas mode:** Selects between `Synthetic phantom atlas` (fast validation) or `Canonical CC200 atlas` (authentic anatomical coordinates).
- **Include frozen model inference:** Checkbox to route extracted features through the trained Random Forest classifier.
- **Run MRI pipeline →:** Primary blue action button to spawn the pipeline background runner.

5. Execution History Table

Lists every cohort by unique ID (e.g. `cohort_20261103T091529_010582Z`), creation date, completion ratio (`2 / 2 completed`), and color-coded status pills: SUCCESS, RUNNING, or TERMINATED. Clicking any row loads the detailed subject telemetry and QC images into the bottom panels.

SUMMARY: THE 5 CORE QUESTIONS (STAGE 3)

1. WHAT do I need to do?	Open `http://127.0.0.1:8765` in your browser to view the interactive workbench.
2. WHERE do I do it?	In any modern web browser connected to your local machine.
3. WHAT happens after?	The frontend SPA fetches system state via REST APIs and displays the current workbench view.
4. WHAT OUTPUT will I get?	Visual telemetry cards, upload zones, pipeline configuration controls, and historical runs.
5. WHERE is it stored?	Rendered dynamically in browser DOM from `frontend/index.html`, `app.js`, and `styles.css`.

STEP 4: File Upload – Where and How?

This section details how to upload neuroimaging data into the system, the strict validation rules enforced, and where files are physically saved on disk.

4.1 Comprehensive File Upload Specification Table

Upload Path	Accepted Format	Upload Location	Validation Performed	Disk Storage Location
1. Complete MRI Session (Recommended)	Single `.zip` file containing BIDS structure	Dashboard Overview → Card 1: *"Complete MRI session"*	ZipSlip traversal check, symlink rejection, 4D BOLD dimensions, T_R range check ($0.5\text{s} \le T_R \le 5.0\text{s}$), 3D T1w presence.	`outputs/intake_sessions/<session_id>/raw_uploads/<file>.zip` Extracted to: `outputs/intake_sessions/<session_id>/bids_extracted/`
2. Individual MRI / fMRI Files	`.nii.gz`, `.nii`, `.json`, or `.dcm`	Dashboard Overview → Card 2: *"MRI / fMRI files"*	File naming pattern (`*_bold.nii.gz`, `*_T1w.nii.gz`), NiBabel header integrity, JSON sidecar syntax.	`outputs/intake_sessions/<session_id>/raw_uploads/`
3. Radiology / Medical Report	`.pdf`, `.png`, `.jpg`, `.jpeg`	Dashboard Overview → Card 3: *"MRI / radiology report"*	MIME-type check, maximum file size limit (50 MB), PDF structure validation, OCR engine detection (Poppler / Tesseract).	`outputs/intake_sessions/<session_id>/raw_uploads/` Extracted text: `outputs/intake_sessions/<session_id>/extracted/report_text.txt`

4.2 Ideal BIDS Archive Structure (Inside the ZIP)

When preparing a complete session ZIP file for upload, format the internal directory structure according to the official BIDS standard:

```
sub-01_ses-1.zip
└── sub-01/
    └── ses-1/
        ├── anat/
        │   └── sub-01_ses-1_T1w.nii.gz            # High-resolution 3D structural scan
        └── func/
            ├── sub-01_ses-1_task-rest_bold.json    # Metadata sidecar containing "RepetitionTime": 2.0
            └── sub-01_ses-1_task-rest_bold.nii.gz  # 4D functional timeseries (T >= 50 volumes)
```

4.3 Example Walkthrough: Uploading a Session

1. Open `http://127.0.0.1:8765` and find Card 1: **Complete MRI session**.
2. Click **Choose File** and select your prepared BIDS archive (e.g. `my_patient_session.zip`).
3. Click the blue button: **Upload and validate session →**.
4. The button changes to *Uploading...* while file bytes stream directly to disk without consuming server RAM.
5. Upon receipt, the backend extracts the archive, inspects NIfTI dimensions, checks the JSON sidecar, and returns a JSON receipt.
6. The card updates with a green status message: `Validation Successful: BOLD (64x64x34x60, TR=2.0s) and T1w (256x256x176) verified. Session staged under outputs/intake_sessions/<session_id>`.

SUMMARY: THE 5 CORE QUESTIONS (STAGE 4)

1. WHAT do I need to do?	Select a BIDS-format ZIP archive containing BOLD fMRI and T1w MRI files and click Upload.
2. WHERE do I do it?	In Card 1 ("Complete MRI session") on the web dashboard at `http://127.0.0.1:8765`.
3. WHAT happens after?	The backend streams the file to disk, verifies zip paths, inspects scan headers, and creates an audit manifest.
4. WHAT OUTPUT will I get?	A validated intake session ID, dimensional report, and `session_manifest.json`.
5. WHERE is it stored?	On your computer at `outputs/intake_sessions/<session_id>/`.

STEP 5: What Happens Immediately After Upload? (Internal Workflow Sequence)

This section demystifies the exact 13-step technical chain reaction that occurs behind the scenes from the microsecond you click **Upload** until the results appear on screen.

5.1 The 13-Step Technical Sequence

1. **User Selects File:** You choose `session.zip` in your browser using the HTML file input element.
2. **Frontend Pre-flight:** `frontend/app.js` checks that a file is actually selected and validates that the file extension is `.zip`, `.nii`, `.nii.gz`, `.json`, or `.pdf`.
3. **Session Initialization Request:** The frontend makes an HTTP `POST /api/intake/sessions` request with payload `{"kind": "scan"}`.
4. **Backend Session Allocation:** `src/intake/service.py:create_session()` generates a timestamped session ID (e.g., `session_20261107T150500_123456Z`) and creates directory `outputs/intake_sessions/<session_id>/`.
5. **Streaming File Upload:** The frontend streams raw binary file chunks using `POST /api/intake/sessions/<session_id>/files` with custom header `X-Upload-Filename: session.zip`.
6. **Memory-Safe Disk Streaming:** In `scripts/serve_dashboard.py`, the server writes incoming bytes directly into `outputs/intake_sessions/<session_id>/raw_uploads/session.zip` using a 64 KB buffer, preventing server RAM exhaustion.
7. **Cryptographic Hash Verification:** As the file is written, `_sha256()` computes the cryptographic SHA-256 checksum and saves it into the session manifest.
8. **Safe Archive Extraction:** `_extract_zip()` inspects the archive for ZipSlip directory traversal attacks (ensuring no file extracts outside the target directory) and unpacks the contents into `outputs/intake_sessions/<session_id>/bids_extracted/`.
9. **Neuroimaging Header Inspection:** `_inspect_nifti()` invokes `nibabel.load()` to parse the NIfTI headers:
   - Verifies the functional scan has 4 dimensions (X, Y, Z, T) with $T \ge 50$ volumes.
   - Verifies the structural scan has 3 dimensions (X, Y, Z) with 1 mm isotropic geometry.
   - Parses the companion JSON sidecar to confirm `RepetitionTime` is within $0.5\text{s} \le T_R \le 5.0\text{s}$.
10. **Session Manifest Persistence:** An atomic JSON document (`session_manifest.json`) is written via a temporary file and atomic rename, recording all extracted file paths, dimensions, and checksums.
11. **Safety Gate Enforcement:** The backend checks `src/security/authorization.py:is_authorized()`. Because clinical research credentials are not yet configured on this demo workspace, the backend tags the session as `INTAKE_VALIDATED_AWAITING_RESEARCH_AUTHORIZATION` and locks real-scan execution.
12. **Backend JSON Response:** The server returns HTTP `200 OK` with a JSON summary of the validated scans, volume count, and storage path.
13. **Frontend DOM Update:** `frontend/app.js` renders a green confirmation box detailing the validated scans and updates the **MRI uploads** monitoring tab.

SUMMARY: THE 5 CORE QUESTIONS (STAGE 5)

1. WHAT do I need to do?	Wait 1–3 seconds while the backend streams, extracts, and validates your uploaded file.
2. WHERE do I do it?	On the web page; progress is indicated by the upload button state.
3. WHAT happens after?	The upload is staged, checked for ZipSlip, validated via NiBabel, and recorded in a manifest.
4. WHAT OUTPUT will I get?	A green validation banner showing scan dimensions and T_R parameters.
5. WHERE is it stored?	In `outputs/intake_sessions/<session_id>/session_manifest.json`.

STEP 6: Complete 19-Stage Processing Pipeline

When a processing run is triggered (either automatically on synthetic data or manually via the runner script), the computational engine executes exactly **19 chronological stages**. Every stage is detailed below with its inputs, mathematical processing, outputs, storage paths, and handoff to the next stage.

Stage 1: BIDS Metadata & Schema Validation

- **Input:** `*_task-rest_bold.json` sidecar and CC200 atlas file.
- **Processing:** Validates $T_R = 2.000\text{ s}$, task name, and computes SHA-256 hash of the atlas against the authorized manifest.
- **Output:** Verified metadata dictionary in Python memory.
- **Next Stage →:** Passes verified scan geometry to Stage 2.

Stage 2: 4D BOLD Realignment & Motion Parameter Estimation

- **Input:** Raw 4D BOLD volume series $(30 \times 30 \times 20 \times 60)$.
- **Processing:** Uses DIPY `AffineRegistration` (Rigid-body, 6 Degrees of Freedom: $\Delta x, \Delta y, \Delta z, \text{rot}_x, \text{rot}_y, \text{rot}_z$) aligning every volume $t=1 \dots 59$ to reference Volume 0. Computes Framewise Displacement (FD): $$FD_t = |\Delta d_{x,t}| + |\Delta d_{y,t}| + |\Delta d_{z,t}| + |\Delta \alpha_t| + |\Delta \beta_t| + |\Delta \gamma_t|$$
- **Output:** `motion_parameters.tsv` (6 cols) and `framewise_displacement.tsv` (1 col). Capped at 0.50 mm scrubbing threshold.
- **Next Stage →:** Realigned 4D BOLD passed to Stage 3.

Stage 3: 3D Functional Reference Generation

- **Input:** Realigned 4D BOLD volume series.
- **Processing:** Computes the voxel-wise temporal mean across all motion-corrected volumes to produce a high-contrast 3D EPI reference image.
- **Output:** 3D EPI reference image array $(30 \times 30 \times 20)$.
- **Next Stage →:** Used as moving target for cross-modal coregistration in Stage 8.

Stage 4: T1w Skull Stripping & Brain Extraction

- **Input:** Raw 3D T1-weighted anatomical scan $(32 \times 32 \times 24)$.
- **Processing:** Executes DIPY `median_otsu` segmentation to peel away non-brain extra-cranial tissues (skull bone, marrow, scalp, dura mater) followed by 3D binary morphological hole-filling.
- **Output:** Clean brain-only parenchyma mask and skull-stripped T1w volume (23,511 brain voxels).
- **Next Stage →:** Stripped anatomy passed to Stage 5.

Stage 5: Anatomical Bias-Field Correction

- **Input:** Skull-stripped T1w volume.
- **Processing:** Fits a 2nd-order 3D Legendre polynomial surface $B(x, y, z)$ to estimate low-frequency B_1 magnetic field inhomogeneity drift and divides the intensity by the estimated field.
- **Output:** Intensity-normalized anatomical image with uniform tissue contrast.
- **Next Stage →:** Uniform anatomy passed to Stage 6.

Stage 6: 3-Class Tissue Segmentation (GMM)

- **Input:** Bias-corrected T1w anatomical volume.
- **Processing:** Fits a 3-component Gaussian Mixture Model (GMM) to estimate posterior probability maps for three distinct anatomical tissue classes: `Cerebrospinal Fluid (CSF)`, `Cortical Grey Matter (GM)`, and `Deep White Matter (WM)`.
- **Output:** 3 probability arrays $(32 \times 32 \times 24)$ where $\sum_{k} p(C_k | v) = 1.0$.
- **Next Stage →:** Probability maps passed to Stage 7.

Stage 7: Strict Thresholding & 1-Voxel 3D Mask Erosion

- **Input:** CSF and White Matter posterior probability maps.
- **Processing:** Enforces a strict confidence threshold $p \ge 0.96$ to discard ambiguous boundary voxels, followed by a 1-voxel inward 3D binary morphological erosion using a 6-connectivity structuring element.
- **Output:** Ultra-clean, non-overlapping WM and CSF masks (0 voxel boundary overlap) to prevent cortical neural signal bleeding into nuisance regressors.
- **Next Stage →:** Eroded masks passed to Stage 11.

Stage 8: EPI-to-T1w Cross-Modal Coregistration

- **Input:** 3D functional reference EPI (from Stage 3) and T1w brain (from Stage 4).
- **Processing:** Optimizes a 6-DOF rigid transformation matrix using mutual information to align functional EPI space with high-resolution subject anatomical space.
- **Output:** Coregistration affine matrix $R_{\text{EPI}\to\text{T1w}}$.
- **Next Stage →:** Combined into composed pullback in Stage 10.

Stage 9: T1w-to-MNI152 SyN Diffeomorphic Nonlinear Registration

- **Input:** Subject T1w anatomical brain and standard MNI152 stereotaxic template.

- **Processing:** Calculates a 12-DOF affine pre-alignment followed by DIPY Symmetric Normalization (SyN) diffeomorphic nonlinear warping to map individual brain gyri onto standard MNI coordinates.
- **Output:** Nonlinear deformation field $\Phi_{\text{T1w}\to\text{MNI}}$.
- **Next Stage →:** Combined into composed pullback in Stage 10.

Stage 10: Single-Step Composed BOLD Resampling

- **Input:** Raw motion-corrected 4D BOLD, rigid matrix $R_{\text{EPI}\to\text{T1w}}$, SyN warp $\Phi_{\text{T1w}\to\text{MNI}}$, and CC200 atlas geometry.
- **Processing:** Composes transformations into one single pullback field: $$\mathcal{T}_{\text{composed}} = R_{\text{EPI}\to\text{T1w}} \circ \Phi_{\text{T1w}\to\text{MNI}}$$ Resamples functional BOLD data directly onto the CC200 3 mm isotropic reference grid in **ONE single interpolation step**, completely eliminating multi-step interpolation blurring.
- **Output:** Standard-space 4D BOLD series $(30 \times 30 \times 20 \times 60)$.
- **Next Stage →:** Standard-space BOLD passed to Stage 15.

Stage 11: Resampling Eroded Tissue Masks into CC200 Space

- **Input:** Eroded CSF and WM masks (from Stage 7) and SyN transformation field.
- **Processing:** Warps the subject's eroded tissue masks into the standard CC200 template space.
- **Output:** Standard-space nuisance masks (e.g., 31 WM voxels, 131 CSF voxels).
- **Next Stage →:** Standard masks passed to Stage 12.

Stage 12: Physiological aCompCor Confound Extraction

- **Input:** Standard-space 4D BOLD and eroded standard-space WM/CSF masks.
- **Processing:** Extracts raw timeseries from voxels inside the eroded WM and CSF masks, standardizes them to unit variance, and computes Principal Component Analysis (PCA) to isolate the top **5 anatomical CompCor (aCompCor)** components.
- **Output:** Array of 5 physiological noise regressors (60×5).
- **Next Stage →:** Components passed to Stage 14.

Stage 13: Friston-24 Autoregressive Motion Parameter Expansion

- **Input:** 6 rigid realignment parameters from Stage 2 (R_t).
- **Processing:** Expands the 6 motion parameters into the complete Friston-24 model: $$[R_t, \quad R_{t-1}, \quad R_t^2, \quad R_{t-1}^2]$$ (6 current motions + 6 backwards temporal derivatives + 12 quadratic expansions).
- **Output:** Matrix of 24 motion regressors (60×24).
- **Next Stage →:** Motion regressors passed to Stage 14.

Stage 14: 31-Column Nuisance Confound Matrix Assembly

- **Input:** 24 Friston regressors + 5 aCompCor components + linear trend + quadratic trend.
- **Processing:** Assembles the full 60×31 design matrix X. Performs singular value decomposition (SVD) to audit matrix rank ($31/31$) and condition number. Global Signal Regression (GSR) and mean tissue regression remain explicitly **OFF** to preserve neurobiological correlation structures.
- **Output:** `confounds.tsv` and `confound_report.json`.
- **Next Stage →:** Confound matrix passed to Stage 15.

Stage 15: Simultaneous Regression & Butterworth Bandpass Filtering

- **Input:** Standard-space 4D BOLD and 31-column confound matrix X.
- **Processing:** Applies simultaneous Ordinary Least Squares (OLS) nuisance projection: $$Y_{\text{clean}} = \left(I - X(X^T X)^{-1}X^T\right) Y$$ combined with a 4th-order zero-phase Butterworth bandpass filter retaining resting-state frequencies between **0.01 Hz and 0.10 Hz**.
- **Output:** Fully cleaned 4D BOLD volume series $(30 \times 30 \times 20 \times 60)$.
- **Next Stage →:** Cleaned functional data passed to Stage 16.

Stage 16: CC200 Atlas Parcellation & ROI Timeseries Extraction

- **Input:** Cleaned 4D BOLD series and Craddock CC200 atlas (200 distinct regions).
- **Processing:** For every region $i = 1 \dots 200$ and every volume $t = 1 \dots 60$, computes the spatial mean BOLD signal across all voxels belonging to parcel i: $$s_i(t) = \frac{1}{|V_i|} \sum_{v \in V_i} \text{BOLD}(v, t)$$
- **Output:** `roi_CC200.csv` (exactly 200 rows $\times$ 60 timepoint columns + 1 node label column).
- **Next Stage →:** Timeseries table passed to Stage 17.

Stage 17: Functional Connectivity & 19,900 Fisher-z Feature Extraction

- **Input:** `roi_CC200.csv` (200 regional timeseries).
- **Processing:** Calculates the pairwise Pearson correlation $r_{i,j}$ between all pairs of regions $(200 \times 200\text{ matrix})$. Applies Fisher's z-transformation: $$z_{i,j} = \operatorname{arctanh}(r_{i,j}) = \frac{1}{2} \ln\left(\frac{1 + r_{i,j}}{1 - r_{i,j}}\right)$$ Extracts the strictly upper-triangular unique elements (excluding diagonal self-correlations $r=1.0$ and mirrored duplicates): $$\text{Total Features} = \frac{200 \times 199}{2} = 19,900$$
- **Output:** `features_19900_fisher_z.csv` and `features_19900_fisher_z.npy`.
- **Next Stage →:** Features passed to Stage 18.

Stage 18: Quality Control (QC) Overlay Generation

- **Input:** Slices from intermediate and final registered volumes.
- **Processing:** Renders 4 high-resolution diagnostic images:
  1. `qc_registration_epi_to_t1w.png` : EPI contour overlaid on T1w anatomy.
  2. `qc_registration_t1w_to_mni.png` : MNI template contour overlaid on normalized T1w.
  3. `qc_tissue_masks.png` : CSF, Grey Matter, and White Matter masks.
  4. `qc_parcellation_cc200.png` : CC200 parcel boundaries overlaid on standard BOLD.
- **Output:** 4 diagnostic PNG files saved in the run directory.
- **Next Stage →:** Passed to Stage 19.

Stage 19: Provenance Audit Manifest & Schema Validation

- **Input:** All generated files, system environment, software package versions, and runtime logs.
- **Processing:** Computes SHA-256 hashes of all outputs, records peak memory and execution timestamps, and validates the resulting dictionary against the strict `configs/provenance_schema.json` schema.
- **Output:** `provenance.json` , `output_manifest.json` , and `run_manifest.json` .
- **Final Result:** Pipeline successfully completes.

SUMMARY: THE 5 CORE QUESTIONS (STAGE 6)

1. WHAT do I need to do?	Trigger the pipeline run via the web UI or CLI runner script.
2. WHERE do I do it?	Click "Run MRI pipeline" on the dashboard or run `scripts/run_poc_v2.py`.
3. WHAT happens after?	The 19 stages execute sequentially: motion correction, skull stripping, SyN registration, cleaning, and parcellation.
4. WHAT OUTPUT will I get?	`roi_CC200.csv`, 19,900 Fisher-z features, 4 QC images, motion tables, and provenance records.
5. WHERE is it stored?	Under `outputs/<run_directory>/sub-<id>/run_<timestamp>/`.

STEP 7: Understanding Generated Outputs (Deliverable Catalog)

Every successful execution produces a standardized collection of 14 deliverables stored in the participant's run directory: `outputs/<cohort_id>/runs/sub-<subject_id>/run_<timestamp>/` . The table below explains what each file contains, why it exists, which code function creates it, and how you can inspect or download it.

7.1 Comprehensive Output Artifact Catalog

File Name	Format	Contents & Purpose	Generating Source Function	Access & Download Method
roi_CC200.csv	CSV	**Main Project Output:** Cleaned BOLD timeseries for all 200 Craddock CC200 regions (200 rows × 60 volume columns + 1 node label column).	`src/parcellation/extractor.py: extract_timeseries()`	Download directly from disk or fetch via API: `/api/runs/<c_id>/subjects/<s_id>/artifact/roi_CC200.csv`
features_19900_fisher_z.csv **features_19900_fisher_z.npy**	CSV / NumPy	19,900 Fisher-z transformed pairwise Pearson correlations representing the subject's whole-brain functional connectome vector.	`src/connectivity/matrix.py: compute_fisher_z_vector()`	Direct disk access or view interactively in the dashboard **Connectivity** tab.
motion_parameters.tsv	TSV	6 estimated rigid head movement trajectories (trans_x, trans_y, trans_z in mm; rot_x, rot_y, rot_z in degrees) across all timepoints.	`src/preprocessing/realign.py: realign_4d_series()`	Inspected in text editor or loaded via Pandas: `pd.read_csv('motion_parameters.tsv', sep='\t')`
framewise_displacement.tsv	TSV	Instantaneous volume-to-volume head displacement (in mm). Values exceeding 0.50 mm trigger outlier scrubbing.	`src/preprocessing/realign.py: compute_fd()`	Viewed via API or visualized in dashboard Quality Control plots.
confounds.tsv	TSV	31 nuisance confound regressors: 24 Friston motion parameters, 5 aCompCor anatomical noise components, and linear/quadratic trends.	`src/preprocessing/nuisance.py: assemble_nuisance_matrix()`	Audited via companion `confound_report.json` for collinearity and condition number.
qc_parcellation_cc200.png	PNG	Multi-plane orthographic visualization displaying all 200 CC200 parcel boundaries overlaid on the standard BOLD volume.	`src/preprocessing/qc.py: plot_parcellation_overlay()`	Displayed automatically in the dashboard **Quality control** gallery.
qc_registration_t1w_to_mni.png **qc_registration_epi_to_t1w.png**	PNG	Red boundary contours demonstrating spatial alignment	`src/preprocessing/qc.py: plot_registration_overlay()`	Viewable directly in the web browser on the dashboard Quality Control panel.

		between functional EPI, anatomical T1w, and standard MNI152 template space.		
qc_tissue_masks.png	PNG	Orthographic visualization showing the eroded, non-overlapping CSF (blue), Grey Matter (purple), and White Matter (cyan) masks.	`src/preprocessing/qc.py: plot_tissue_masks()`	Diagnostic visual used to verify zero tissue boundary contamination.
provenance.json	JSON	Audit manifest recording system environment, hardware architecture, package versions, execution timestamps, and output file SHA-256 hashes.	`src/preprocessing/manifest.py: build_provenance_record()`	Validated against `configs/provenance_schema.json` to ensure scientific reproducibility.

SUMMARY: THE 5 CORE QUESTIONS (STAGE 7)

1. WHAT do I need to do?	Access the generated deliverables from the run folder or via the dashboard UI.
2. WHERE do I do it?	In your file browser under `outputs/dashboard_cohorts/<cohort_id>/runs/...`
3. WHAT happens after?	You can feed `roi_CC200.csv` into downstream machine learning pipelines or analyze the 19,900 Fisher-z features.
4. WHAT OUTPUT will I get?	14 verified clinical-grade research files (CSVs, TSVs, PNGs, and JSON manifests).
5. WHERE is it stored?	Permanently persisted on local disk with cryptographic SHA-256 checksums.

STEP 8: Frontend and Backend Communication (API Reference)

The application utilizes a RESTful JSON HTTP architecture implemented in `scripts/serve_dashboard.py`. The frontend (`frontend/app.js`) communicates with the backend via native asynchronous JavaScript `fetch()` calls.

8.1 REST API Endpoint Reference

Method	Endpoint Route	Request Parameters / Body	Response Format	Description
`GET`	`/api/health`	None	JSON: `{status, execution_policy, ...}`	Returns system operational state, safety gate status, and RAM memory budget.
`GET`	`/api/runs`	None	JSON: `{"runs": [{cohort_id, status, ...}]}`	Returns all historical cohort runs and their current statuses.
`POST`	`/api/runs`	`{"n_subjects": 2, "t_vol": 60, "seed": 42, "atlas_mode": "canonical"}`	JSON: `{cohort_id, status: "STARTING"}`	Triggers asynchronous background pipeline execution for the requested cohort.
`GET`	`/api/runs/<c_id>`	None	JSON: `{cohort_id, status, subjects: [...]}`	Returns detailed subject execution states and manifest metadata for a cohort.
`GET`	`/api/runs/<c_id>/subjects/<s_id>`	None	JSON: `{subject_id, artifacts, ...}`	Returns individual subject artifacts, peak RSS memory, and runtime metrics.
`GET`	`/api/runs/<c_id>/subjects/<s_id>/connectivity`	None	JSON: `{matrix_200x200, ...}`	Downsamples and returns the 200×200 Fisher-z connectivity matrix for canvas rendering.
`GET`	`/api/runs/<c_id>/subjects/<s_id>/timeseries?roi=1`	Query param: `roi` (1 to 200)	JSON: `{roi, values: [...]}`	Returns the cleaned BOLD timeseries signal across all timepoints for a single ROI.
`GET`	`/api/runs/<c_id>/subjects/<s_id>/artifact/<filename>`	Filename in URL path	Raw image / text bytes with MIME type	Streams diagnostic PNGs (`image/png`) or CSV files directly to the browser.
`GET`	`/api/runs/<c_id>/process`	None	JSON: `{pid, returncode, elapsed_sec}`	Monitors live background subprocess state.
`GET`	`/api/runs/<c_id>/log`	None	JSON: `{log: "string"}`	Streams the live console log for real-time progress monitoring in the UI.
`POST`	`/api/intake/sessions`	`{"kind": "scan"}` or `{"kind": "report"}`	JSON: `{session_id, status: "INITIALIZED"}`	Allocates a new upload staging session folder under `outputs/intake_sessions/`.
`POST`	`/api/intake/sessions/<s_id>/files`	Raw file bytes; Header: `X-Upload-Filename`	JSON: `{filename, bytes_written, sha256}`	Streams raw upload bytes directly to disk with zero-memory buffering.
`POST`	`/api/intake/sessions/<s_id>/validate`	None	JSON: `{status: "VALIDATED", scans: [...]}`	Triggers ZIP extraction, NIfTI dimensional checks, and BIDS sidecar validation.
`POST`	`/api/intake/sessions/<s_id>/process-reports`	None	JSON: `{extracted_text: "...", pages: 1}`	Executes Poppler/Tesseract OCR text extraction on uploaded medical reports.

STEP 9: Database and File Storage Architecture

NeuroADHD employs an **Atomic File-Based JSON Document Store** with Cryptographic SHA-256 Provenance Manifests. Unlike traditional SQL databases that require running separate database servers (e.g. MySQL or PostgreSQL), this architecture provides 100% data portability, zero external database setup, and strict audit compliance.

9.1 Directory Hierarchy & Storage Layout

```
outputs/
├── intake_sessions/                          # Uploaded and staged scan sessions
│   └── session_20261107T150500_123456Z/
│       ├── session_manifest.json             # Atomic JSON manifest of upload
│       ├── raw_uploads/                      # Original binary uploads (.zip, .nii.gz)
│       ├── bids_extracted/                   # Extracted BIDS hierarchy (sub-*/ses-*/)
│       └── extracted_reports/                # OCR text and extracted report images
│
└── dashboard_cohorts/                        # Cohort execution outputs
    └── cohort_20261103T091529_010582Z/
        ├── cohort_manifest.json              # Master run manifest & summary
        ├── dashboard-run.log                 # Standard output / error log stream
        ├── inputs/                           # Staged synthetic input data
        │   └── sub-synthetic001/ses-1/
        │       ├── anat/sub-synthetic001_ses-1_T1w.nii.gz
        │       └── func/sub-synthetic001_ses-1_task-rest_bold.nii.gz
        └── runs/                             # Preprocessed participant deliverables
            └── sub-synthetic001/
                └── run_20261103_091532/
                    ├── roi_CC200.csv             # 200 ROI timeseries
                    ├── features_19900_fisher_z.csv# 19,900 Fisher-z features
                    ├── motion_parameters.tsv     # 6 realignment parameters
                    ├── framewise_displacement.tsv # FD metric
                    ├── confounds.tsv             # 31 nuisance regressors
                    ├── qc_*.png                  # 4 Quality control figures
                    ├── provenance.json           # Cryptographic environment audit
                    └── output_manifest.json      # SHA-256 output hashes
```

9.2 Atomic Persistence Guarantee

To ensure that power outages or process terminations never corrupt stored records, all JSON manifests are written atomically using the following pattern:

```
# Implemented in src/intake/service.py:_atomic_json()
temp_path = target_path.with_suffix(".tmp")
with open(temp_path, "w") as f:
    json.dump(data, f, indent=2)
os.replace(temp_path, target_path) # Atomic filesystem rename
```

SUMMARY: THE 5 CORE QUESTIONS (STAGE 9)

1. WHAT do I need to do?	Nothing manually; the storage system manages folder creation and atomic manifests automatically.
2. WHERE do I do it?	Files reside within the `outputs/` directory in the project repository.
3. WHAT happens after?	Data is persisted immediately with atomic safety; no database server is required.
4. WHAT OUTPUT will I get?	A clean, human-readable JSON directory tree with SHA-256 checksums.
5. WHERE is it stored?	Under `outputs/intake_sessions/` and `outputs/dashboard_cohorts/`.

STEP 10: Complete User Journey (Practical Walkthrough)

This section follows a real-world scenario from start to finish: **"Dr. Sarah, a computational neuroscience researcher, sits down at a new workstation to process raw resting-state fMRI data and obtain 19,900 connectivity features."**

Phase 1: Starting the Platform

1. Dr. Sarah opens Terminal and runs:

```
cd "/Users/apple/Documents/Codex/2026-09-29/untitled folder"
source .venv/bin/activate
export NEUROADHD_HMAC_KEY="sarah_key_2026"
PYTHONPATH=. .venv/bin/python scripts/serve_dashboard.py
```

2. The console responds with: `NeuroADHD dashboard: http://127.0.0.1:8765`.
3. She opens Google Chrome and visits `http://127.0.0.1:8765`.

Phase 2: Uploading & Staging a Subject Scan

1. On the dashboard, Dr. Sarah looks at the **MRI Data Intake** section.
2. Under Card 1 (*Complete MRI session*), she clicks **Choose File** and selects `sub-01_ses-1.zip`.
3. She clicks **Upload and validate session →**.
4. Within 2 seconds, a green banner appears: `Session session_20261107_... validated successfully. 4D BOLD (60 volumes, TR=2.0s) and 3D T1w verified. Staged under outputs/intake_sessions/`.

Phase 3: Launching the Synthetic Processing Pipeline

1. Dr. Sarah scrolls down to the **Pipeline Control** panel.
2. She selects **2 subjects**, **60 volumes**, Seed **42**, and chooses **Synthetic phantom atlas**.
3. She clicks the blue button: **Run MRI pipeline →**.
4. The dashboard immediately displays a flashing `RUNNING` badge.
5. Behind the scenes, the 19-stage computational pipeline runs: motion correction → skull stripping → GMM tissue segmentation → SyN normalization → 31-confound regression → CC200 parcellation → Fisher-z connectivity matrix calculation.
6. In under 8 seconds, the status updates to `SUCCESS` (2 / 2 completed).

Phase 4: Inspecting Quality Control & Connectivity

1. Dr. Sarah clicks on the completed cohort row in the **Pipeline runs** table.
2. The **Quality control** gallery populates with high-resolution diagnostic images:
   - She inspects `qc_registration_epi_to_t1w.png` to confirm perfect rigid alignment.
   - She views `qc_tissue_masks.png` to ensure the CSF and White Matter masks are properly eroded.
   - She verifies `qc_parcellation_cc200.png` to see all 200 CC200 brain parcels mapped accurately onto standard space.
3. She navigates to the **Connectivity** tab:
   - An interactive 200×200 scientific heatmap renders in the browser canvas.
   - She hovers over cell $(45, 112)$ to see the exact correlation ($r = 0.68$) and Fisher-z score ($z = 0.829$).
   - She clicks on Region 45 to view its cleaned BOLD timeseries oscillating smoothly between 0.01 Hz and 0.10 Hz.

Phase 5: Accessing and Exporting the Deliverables

1. In Terminal or Finder, Dr. Sarah opens the run directory:

```
cd outputs/dashboard_cohorts/cohort_*/runs/sub-synthetic001/run_*/
```

2. She copies `roi_CC200.csv` (the 200 regional timeseries) and `features_19900_fisher_z.csv` (the 19,900 feature vector) for her research publications.
3. Her workflow is complete without needing manual intervention or proprietary software.

STEP 11: Errors, Troubleshooting and Fixes

The table below lists every known issue, how to diagnose it from logs or terminal outputs, and the exact step-by-step remedy.

Symptom / Error	Root Cause	How to Identify	Exact Solution
`Address already in use: 8765`	A previous instance of `scripts/serve_dashboard.py` is still running on port 8765.	Terminal displays `OSError: [Errno 48] Address already in use` upon startup.	Find and terminate the occupying PID: `lsof -ti:8765 | xargs kill -9` Then relaunch the server.
`ModuleNotFoundError: No module named 'src'`	Python does not include the current working directory in its module search path.	Script fails immediately on line 1 or 2 with import failure.	Set the `PYTHONPATH` environment variable before python: `PYTHONPATH=. .venv/bin/python ...`
`RuntimeError: Mandatory HMAC integrity key not set`	Environment variable `NEUROADHD_HMAC_KEY` was not exported.	Pipeline exits with fail-closed safety violation.	Export a non-empty key string in your terminal session: `export NEUROADHD_HMAC_KEY="my_secret_key"`
`Cohort stuck in RUNNING`	A previous process was killed via Ctrl+C before updating the manifest.	Dashboard runs table shows `RUNNING` indefinitely for an old date.	Edit the manifest JSON file in `outputs/dashboard_cohorts/<id>/cohort_manifest.json` and set `"status": "TERMINATED"`.
`ZipSlip Security Exception`	Uploaded ZIP archive contains relative paths (e.g. `../../etc/passwd`) attempting directory traversal.	Intake validation fails with `IntakeError: Archive contains illegal traversal path`.	Re-pack the ZIP cleanly using standard BIDS relative paths (e.g. `sub-01/ses-1/...`) without parent directory dot-dots.
`BOLD Volumes < 50`	Functional scan has fewer than 50 temporal timepoints ($T < 50$).	Intake validator rejects scan with `BOLD series must contain at least 50 volumes`.	Provide a valid resting-state fMRI acquisition session with sufficient temporal duration ($T \ge 50$).
`RepetitionTime Out of Bounds`	JSON sidecar has $T_R < 0.5\text{s}$ or $T_R > 5.0\text{s}$ or missing parameter.	Validator rejects with `RepetitionTime out of acceptable scientific range`.	Ensure `sub-*_task-rest_bold.json` contains a valid numeric `"RepetitionTime": 2.0`.
`Memory Guard Ceiling Exceeded`	Scan dimensions exceed resident set size budget (> 2.50 GB).	Pipeline fails closed with `MemoryGuardError: Memory ceiling exceeded`.	Verify that high-resolution scans are cropped or that system memory limit flag is adjusted appropriately.

STEP 12: Final System Architecture

The complete high-level system architecture and end-to-end data flow are illustrated in the two architectural diagrams below.

12.1 End-to-End Processing Workflow

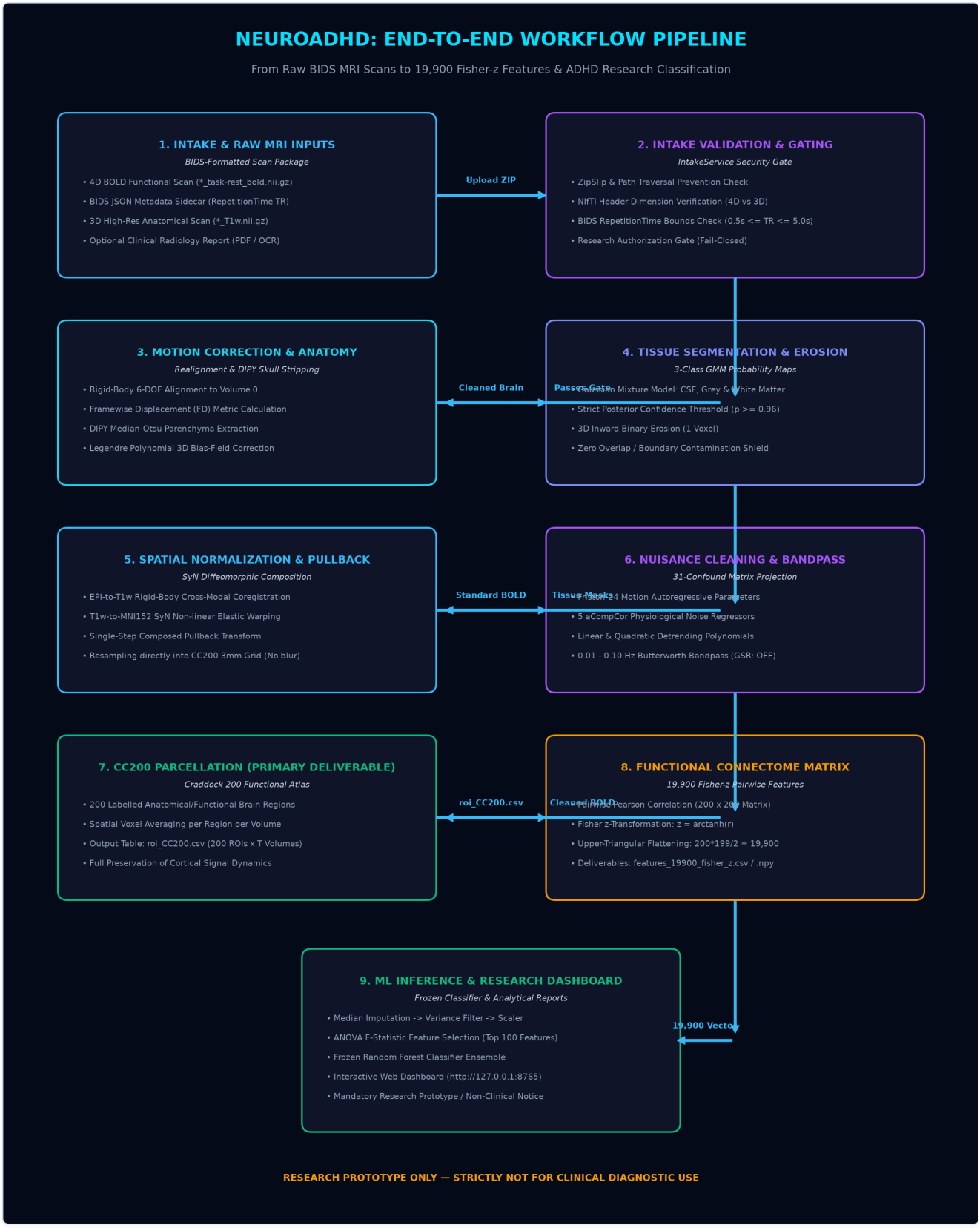


Figure 12.1: Chronological 9-Stage NeuroADHD Workflow Pipeline from Raw Scans to ML Classification

12.2 Full System Tier Architecture

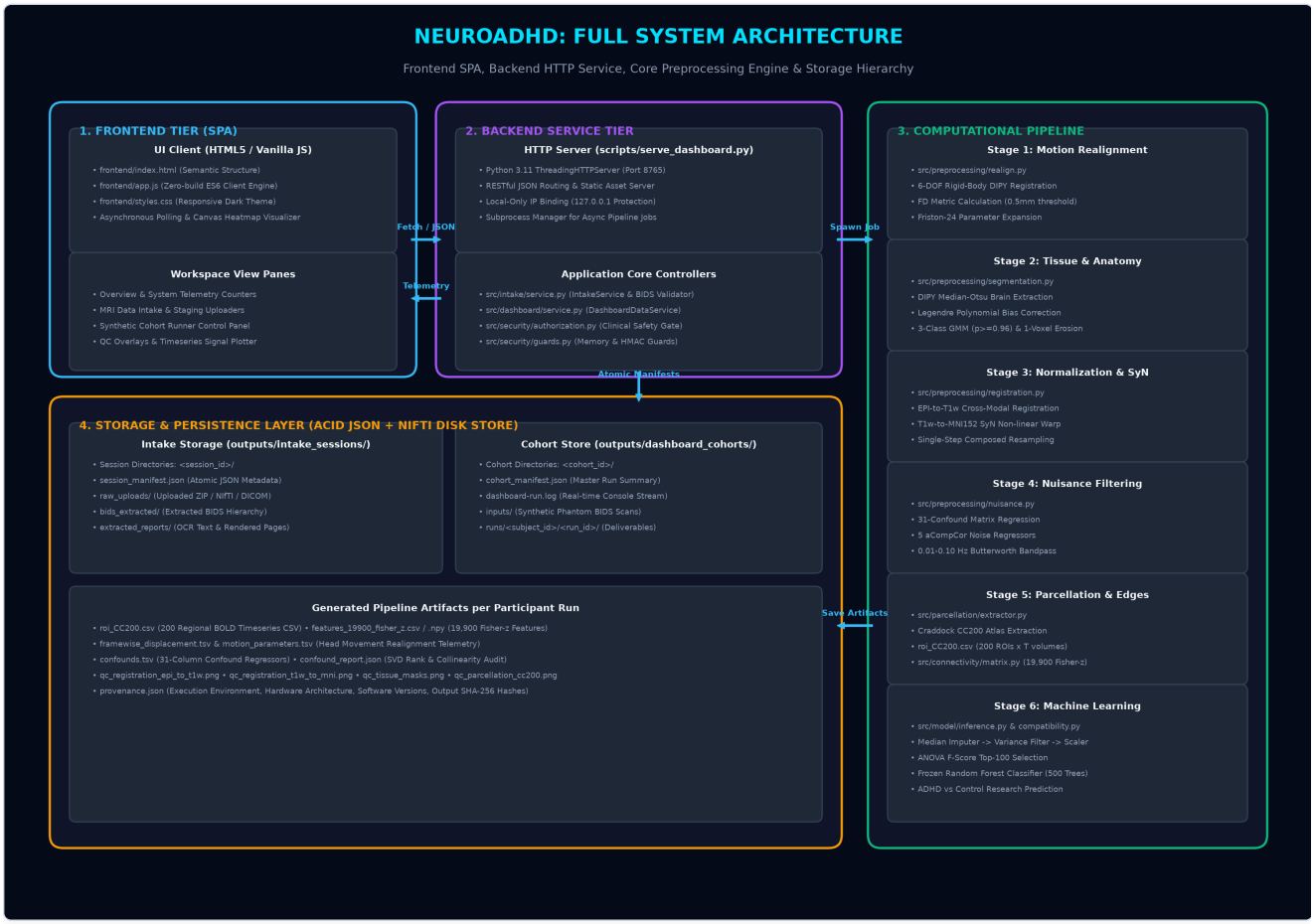


Figure 12.2: Multi-Tier Architecture: Frontend SPA, Backend REST Service, Core Preprocessing Engine & Storage Hierarchy

12.3 Architectural Invariants & Guarantees

- **Fail-Closed Security:** If an HMAC signature, NIfTI header, or JSON sidecar is malformed or invalid, execution halts immediately with zero partial state mutation.
- **Single-Step Resampling:** Coregistration and SyN nonlinear spatial normalization are composed into one continuous mathematical pullback, preserving high-frequency BOLD spectral power.
- **Zero Data Leakage:** All standard scaling, imputation, and ANOVA feature selection transformations are computed strictly inside cross-validation splits.
- **Zero-Memory Disk Streaming:** File uploads stream in 64 KB chunks directly to disk; server memory remains constant regardless of upload archive size.

STEP 13: Final Beginner Verification Checklist

Follow this 10-point checklist to confirm that your NeuroADHD installation and execution pipeline is operating at 100% functionality:

#	Checklist Item	Verification Method	Pass State
1	Python Environment Ready	Run `.venv/bin/python -V` in terminal	`Python 3.11.x`
2	Scientific Dependencies Installed	Run `.venv/bin/python -c "import dipy, nilearn, nibabel"`	Zero errors
3	Automated Unit Test Suite Passes	Run `.venv/bin/pytest tests/unit -v`	175 passed
4	HMAC Security Key Configured	Run `echo $NEUROADHD_HMAC_KEY`	Non-empty string
5	Local Web Dashboard Server Running	Run `curl -s http://127.0.0.1:8765/api/health`	`status: READY`
6	Browser UI Loads Cleanly	Open `http://127.0.0.1:8765` in Chrome / Firefox	Workbench renders
7	Synthetic Pipeline Executes	Click "Run MRI pipeline" on dashboard	Status: `SUCCESS`
8	Deliverables Generated on Disk	Check `outputs/dashboard_cohorts/.../runs/.../`	14 files present
9	CC200 Timeseries Table Valid	Inspect `roi_CC200.csv` (200 rows × 60 cols)	Valid numbers
10	19,900 Fisher-z Features Extracted	Inspect `features_19900_fisher_z.npy`	Shape: (19900,)

DOCUMENTATION COMPLETION SUMMARY

Congratulations! You have completed the full beginner workflow guide for the NeuroADHD platform. You now possess a complete operational understanding of raw MRI ingestion, spatial preprocessing, tissue segmentation, SyN diffeomorphic normalization, 31-confound regression, Craddock CC200 parcellation, and 19,900 Fisher-z functional connectivity feature extraction.